

School of Mechanical and Manufacturing Engineering

Faculty of Engineering

UNSW Sydney

BY

Yiting Liang

Quadcopter Charging/Docking Station with Auto Landing Details

Submitted: August 7, 2026

Student zID: z5611900

Supervisor: Mohammad Deghat, Mitchell Torok

Abstract

Autonomous docking and charging for unmanned aerial vehicles in a dynamic and unstructured environment are needed. This paper will examine the recent progress in UAV automatic docking and power-charging in detail, and two specific parts have been selected for close study: spot detection means accurate and stable operation of landing. Based on the literature review, several deficiencies in the current study have been identified: a lack of environmental robustness under extreme conditions or coupled disturbances; dependence on external positioning infrastructure instead of integrating learning-based controllers with low-cost on-board sensors; and limited adaptability to high-speed dynamic targets with complex trajectories. Based on the above deficiencies, this project will propose several control strategies, verify them in Gazebo simulation and real-world experiments, and integrate the best-performing strategy. The research provides a low cost, on board only, fully autonomous docking system for the quad rotor aircraft. Thus, the quadrotor will be able to operate in a dynamic environment in practice.

Nomenclature

- RMS=root mean square
- EKF=Extended Kalman Filter
- UAV=Unmanned Aerial Vehicle
- UGV=Unmanned Ground Vehicle
- PID=Proportional-Integral-Derivative
- SMC=Sliding Mode Control
- MPC=Model Predictive Control
- RL=Reinforcement Learning
- NDOB=Nonlinear Disturbance Observer

1 Introduction

Quadcopter Unmanned Aerial Vehicles (UAVs) are now widely used by all sections of the public security force for patrol, observation, environmental monitoring, construction supervision and delivery. They are versatile and can reach difficult places, so many people have used them, but the battery life is short, and they cannot operate continuously. Manual battery replacement or charging is slow and labour-intensive, thus not suitable for prolonged autonomous operation. Therefore, in light of the above shortcomings, it should be pointed out that integrated systems comprising autonomous landing and automated recharging have gradually appeared as essential support facilities for extended UAV flight operations. Design a control strategy for autonomous landing of a quadcopter at a docking station in this project. Having achieved the above objectives, there will be no longer a need for humans in the critical post-mission workflow, and quadcopters will be able to operate continuously in both static and dynamic environments. Autonomous landing has to be on point in terms of where you land the quadcopter, take the diference into consideration, and stabilize those controls to make sure that it actually goes straight up to the docking station. There are disturbances, and the wind and sensors cause errors. The project will use the following existing equipment: quadcopters, batteries, a simple simulator and real flight controls (landing is not supported). Key technical enablers are ROS 2 for modular software integration and rapid prototyping. Figure 1 is the workflow of the project, which includes quadcopter navigation, autonomous landing, docking alignment, automatic charging and signal feedback. The parts listed above create a whole, self-sufficient solution that enhances the operating efficiency and utility of quadcopters in actual operation.

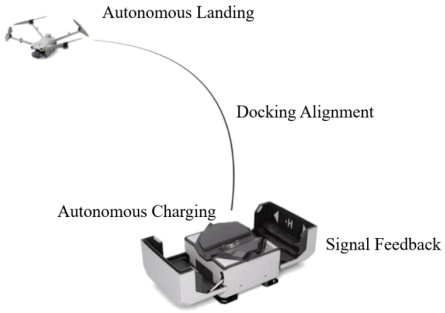


Figure 1: Project's overall workflow, illustrating the integration of autonomous landing, docking, charging, and signal feedback processes.

As shown in Figure 1, the quadcopter will move to a predetermined docking station according to the pre-set waypoints and its present location in real time, and then, using the control method proposed by this project, will automatically land in a position that is perfectly aligned with the charging contacts of the docking station. Alignment is complete, and then the system enters safe charging mode; upon full charge of the battery, a clear finish signal is sent to the ground control computer, which triggers the next mission schedule or standby mode. Next are the reviews of related existing autonomous landing and docking/charging technology literature, followed by the detailed research methodology, result and conclusions. Together, they have provided a good foundation for the successful realisation of the project goals. It should be noted that this paper mainly focus on autonomous landing and docking control, the charging and signal feedback components are included as a conceptual framework for future work.

2 Literature Review

2.1 Autonomous Landing Technology

Autonomous landing is needed before the quadcopter docks and charges; therefore, high-precision positioning, stable control and uncertainty handling must be realised. Research on Position Systems and

Control Algorithms investigates the trade-off among accuracy, reliability and computation speed.

2.1.1 Precise Positioning Systems

Precision Positioning Systems for autonomous landing are divided into two types: vision-based and multi-sensor fusion systems, as shown in Figure 2.

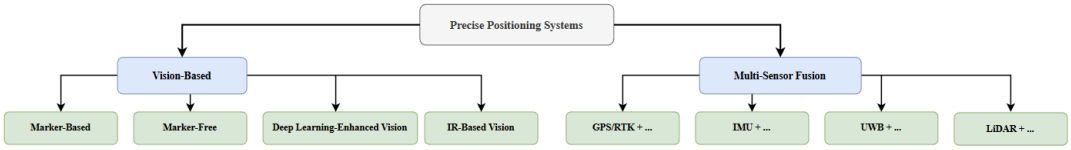


Figure 2: Classification of Precision Positioning Systems for Autonomous Quadcopter Landing

Vision-based systems include marker-based, marker-free, deep-learning-enhanced, and IR-based types. A system of several sensors is used to solve the problems of individual sensors.

Vision-Based Positioning Systems Vision-based Positioning Systems are relatively simple, low-cost devices that do not require external infrastructure such as GPS or motion-capture systems.

1. **Marker-Based Vision Positioning**: Uses pre-defined markers (ArUco, AprilTag) for pose estimation. [1] integrates a downward camera and IMU to achieve X/Y errors of less than 5 cm. Marker-based systems need an organised environment and cannot work in places without markers.

2. **Marker-Free Vision Positioning**: Utilises natural features (e.g., ORB features, SLAM) for localisation in unstructured environments. [2] has achieved <45 cm accuracy and robustness to 3-8 m/s wind, but is constrained by the assumption of ground planarity. [3] applies ORB-SLAM2 for safe landing zone selection in unknown environments; however, sparse point clouds and barometer errors remain problems. [4] constructs terrain maps using RMFPP but employs offline optimisation that is not suitable for real-time onboard processing.

3. **Deep Learning-Enhanced Vision Positioning**: Improves robustness to occlusion and light variations. [5] presents YOLOv4-tiny for real-time docking, but no quantified docking position control accuracy. [6] achieves 92.078% tracking in maritime environments with enhanced YOLOv7.

4. **IR-Based Vision Positioning**: Uses infrared beacons and cameras for stable detection in low-visibility or high-glare conditions. [7] achieved an 18.6cm landing error with IR beacons. [24] reported a 35mm positioning error for Lighthouse IR positioning, but the most precise methods are still too large and not suitable for lightweight UAVs.

Multi-Sensor Fusion for Enhanced Localization Multi-sensor Fusion for Improved Localization: Multi-sensor fusion combines vision with GPS/RTK, IMU, UWB and LiDAR for stable positioning.
[8] shows a three-stage fusion method (UWB + AprilTag + IMU + encoder) and achieves an RMS error of < 0.016m in GPS-denied areas. [9] uses an LKF to integrate GPS, IMU and AprilTag; although the position error is 7.97 cm, it lacks redundancy as it only employs GPS and AprilTag. [9], [8] do dynamic landings but their system relies on complicated MPC or sensor fusions which can be very computationally expensive for a quadcopter without much resources.
LiDAR improves altitude estimation; [10] has achieved an altitude error of < 0.069m. Though is limited by GPS precision and low-visibility vulnerability, the combination of LiDAR and vision has proven effective for navigating complex terrain and avoiding obstacles during landing.

2.1.2 Control Algorithms for Stable Landing

Control Algorithms Convert Positioning Data into Flight Commands. Figure 3 is a division of these algorithms.

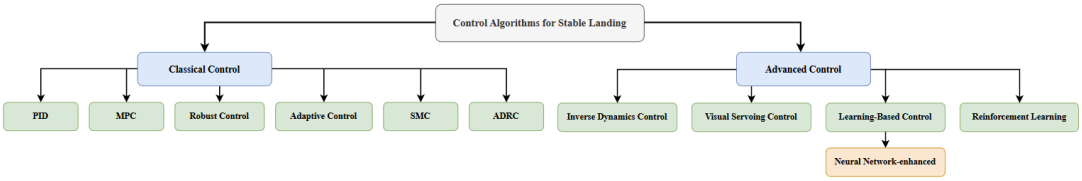


Figure 3: Classification of Control Algorithms for Stable Landing

Classical Control

- **PID**: Classical Control PID controllers are widely used for their simplicity. [1] achieves simulated landing error < 5 cm with vision-based PID. However, PID struggles with nonlinear dynamics and disturbances. [11] proposes NN-PID for wind compensation and slope landing, achieving roll/pitch errors < 0.5° on 10° slopes, though sensor noise robustness is unvalidated.
- **MPC**: Solves online optimisation of a finite horizon for constraints and dynamic objectives. [9] uses linear MPC with integral compensation, achieving a landing error of < 0.37 m on 4-12 m/s platforms, but is simulation-only and not suitable for aggressive trajectories.
- **SMC**: Keeps the system in a state on a sliding surface under uncertainty. [12] combines SMC with LADRC for a faster response (altitude: 2s, yaw: 0.3s) but has only been validated in simulation with ideal state inputs. [13] develops a double closed-loop ADRC with virtual control variables and achieves < 0.1m position errors in simulation, but landing-related accuracy and real-world applicability are unvalidated.

Advanced Control

- **DOB**: Estimates and compensates for external disturbances [7]; achieves ¡7% disturbance estimation error on moving targets (< 2 m/s), but has difficulties with 3D targets and fast-varying disturbances.
- **MINCO**: Optimizes spatiotemporal trajectories with minimum control effort [14], supports dynamic target tracking up to 8.3 m/s and 20 Hz replanning, but depends on external sensors (GPS/visual) and is susceptible to occlusion.
- **Visual Servoing Control**: Visual Servoing Control uses image feedback directly for control [15] and achieves robust moving-target landing in the presence of 8-m/s wind, but suffers from a 60-100ms latency. RGB-Saturation fusion with enhanced YOLOv7 is employed in [6] to achieve 92.078% tracking accuracy, improves small target detection, but computational complexity increases with model enhancements.
- **Learning-Based Control**: RL and NN-based controllers offer adaptability to complex dynamics. [16] reviews DRL controllers achieving < 0.1 m landing error, but with simulated sensors for DRL training, real-world sensor latency/noise and sim-to-real mismatch affect accuracy; and [17] proposes SN-DNN for ground effect compensation, reducing z-axis error from 0.153 m to 0.027 m, but relies on motion capture systems and large offline datasets.

2.2 Docking Station and Disconnection Mechanism Design

2.2.1 Docking Hardware Design

Docking Mechanisms need to ensure accurate alignment, stable connection and compatibility with charging. Key Designs include:

- **Magnetic-Aided Docking**: [18] employs electropermanent magnets (EPMs) with a 140 N attraction and servo locking, and 3D-printed guides and torsion springs for inclined contact.
- **Passive Mechanical Guided Docking**: [19] uses landing pads with central markers and mechanical alignment. [14] employs Velcro fastening for high-speed perching but is limited by surface flatness.

2.2.2 Disconnection Mechanism

Disconnection mechanisms balance reliability and low wear:

- **Active mechanical pushrod**: [20] is RFID-guided alignment and pushrods. [11] is a 2-link landing leg for slope adaptation up to 20°.
- **Non-contact magnetic release**: [18] Demagnetizes EPMs and releases a servo-driven locking lever for smooth, low-impact separation.

2.3 Automated Charging Systems

Automatic Charging Stations can automatically add power to vehicles.

2.3.1 Charging Methods

- **Contact Charging**: relatively simple but requires high-precision alignment; [21] uses spring contacts and balance chargers with overcharge protection.
- **Wireless Charging**: No physical contact and more durable. Inductive coupling has high efficiency and a long range; capacitive power transfer is light but less efficient [20], and therefore XT-60 contact charging is employed as a fail-safe.

2.3.2 Full-Charge Signal Feedback

Full-charge detection based on voltage and current monitoring:

- **Voltage threshold**: [20] is for full-charge feedback using voltage monitoring; [22] is a hybrid system with SOC sensors.
- **Current threshold**: [23] detects trickle current and triggers full-charge cutoff.

2.4 Critical Research Gaps

Three consistent deficiencies have been identified in the reviewed literature on this paper that restrict actual application:

1. **Environmental Adaptability is lacking**: Most precision landing systems are based on visual markers or infrared signals, which fail in extreme environments (low light, fog, rain, dust); although multi-sensor fusion has been applied to solve the problem of GPS denial, low-cost, lightweight on-board solutions for indoor or urban canyon scenarios are still unavailable; furthermore, few studies have verified performance under coupled disturbances (wind + ground effect/slope) typical of outdoor operation, leaving the system vulnerable to unmodeled real-world interference.

2. **Perception and localisation are relatively dependent on external infrastructure**: high-precision landing results generally require noise-free state estimation provided by motion-capture systems such as Vicon and OptiTrack, but the setup validation process for using only on-board sensors is lacking, although these setups are necessary for field application. Meanwhile, learning-based controllers (such as DRL and NN-PID) have shown promise in simulations, but are rarely integrated with low-cost airborne sensors, and their robustness to real-world sensor noise, latency, or drift remains unproven.

3. **Dynamic target adaptation is lacking**: The existing moving-target landing systems are limited to low-speed, linear trajectories, without validation for high-speed targets or aggressive maneuvers, and control logics such as PID or fixed-horizon MPC cannot respond quickly to rapid changes in trajectory, resulting in tracking lag and misalignment.

2.5 Project Justification

Based on the above shortcomings of this project, an on-board solution will be developed. The way of doing so uses the existing technology and puts forward the following four main improvements:
1. To build a strong control scheme that accounts for disturbances in a coupled environment.
2. Use a few on-board sensors to obtain some positional data in an unstructured environment.
3. To Design and Validate a Control System for Difficult Targets to Close the Gap Between Simulation and Application.
4. Landing controllers need to be well-connected, and at all times during the workflow, the docking/charging device and a reasonable signal must be returned.
The above optimisations address the main research deficiencies pointed out in the review of previous studies and enhance the robustness of the system to real-world outdoor environments.

3 Methodology

3.1 Research Question

Given the demand for autonomous landing and charging of quadcopters, this project will tackle two main technical issues and put forward the following research questions: Design a landing control method that can cope with environmental disturbances (such as wind and ground effect) and sensor noise to achieve accurate, autonomous docking of the quadcopter with a docking station.

3.2 Research Aims

To develop different types of landing control strategies based on ROS 2, and test their performance in Gazebo simulations (disturbances, noise) and real-world flights.

3.3 Proposed Solution and Experimental Methodology

To achieve the main goals of this project, the following technical solution will be introduced in this section: (1) A vision-based perception module to estimate the relative pose and velocity of the ArUco marker with respect to the drone; (2) An NDOB model for estimating external wind and other disturbances; (3) A hybrid adaptive controller that combines adaptive SMC for altitude control and multi-variable state feedback with predictive integral and wind compensation for horizontal position control.

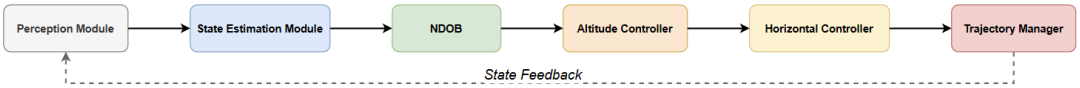


Figure 4: Overview of the proposed control architecture, illustrating the information flow from perception to actuation.

The six modules of the architecture are Perception, State Estimation, NDOB, Altitude Controller, Horizontal Controller and Trajectory Manager, which are connected in a cascaded manner as shown in Figure 4.

The whole system pipeline is as follows: read pose → extract marker relative velocity → compute visual error → estimate wind → calculate control outputs → publish ELRS flight commands → log data.

3.3.1 Coordinate Frames and System Overview

The drone's full state in the world inertial frame $\mathcal{F}_w$ is represented as:

$$\mathbf{x} = \begin{bmatrix} x & y & z & \dot{x} & \dot{y} & \dot{z} & q_w & q_x & q_y & q_z \end{bmatrix}^T \in \mathbb{R}^{10} \tag{1}$$

where $\mathbf{p} = [x, y, z]^T$ and $\dot{\mathbf{p}} = [\dot{x}, \dot{y}, \dot{z}]^T$ are the position and velocity in the inertial frame, and $\mathbf{q} = [q_w, q_x, q_y, q_z]^T$ is the unit quaternion representing the drone's attitude. The quaternion is converted to Euler angles (ϕ, θ, ψ) for control computation. And internal state remains stored in quaternion form to avoid singularities.

3.3.2 Perception and Localization via ArUco Markers

Marker Detection A vision system that uses ArUco markers (dictionary: `DICT_5X5_1000`)) for relative positioning. A downward-facing monocular camera publishes ROS image topics and processes them in an independent background thread to avoid control-loop latency. Given a detected marker, the pose estimation problem is set up as:

$$\mathbf{R}_m, \mathbf{t}_m = \arg\min_{\mathbf{R},\mathbf{t}} \sum_{i=1}^{4} \|\mathbf{p}_i - \mathbf{K}\,[\mathbf{R} \mid \mathbf{t}]\,\mathbf{P}_i\|^2 \tag{2}$$

where $\mathbf{p}_i \in \mathbb{R}^2$ are the detected corner pixel coordinates, $\mathbf{P}_i \in \mathbb{R}^3$ are the corresponding 3D corner points in the marker's local coordinate frame, $\mathbf{K} \in \mathbb{R}^{3\times3}$ is the intrinsic camera matrix, and $\mathbf{R}_m \in SO(3)$, $\mathbf{t}_m = [t_x, t_y, t_z]^T \in \mathbb{R}^3$ are the rotation matrix and translation vector of the marker relative to the camera frame.

Use the calibrated intrinsic parameters to calculate the Camera Matrix as follows:

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 245.18 & 0 & 327.82 \\ 0 & 328.95 & 240.98 \\ 0 & 0 & 1 \end{bmatrix} \tag{3}$$

To address the problem of marker occlusion at close range, a nested dual-marker strategy has been employed. As shown in Figure 5, a smaller ArUco marker (ID 1) is embedded in the centre of a larger marker (ID 0), and a white isolation border has been added to prevent detection interference. The larger marker (ID 0) is used for coarse positioning at a higher altitude of the drone, and the full marker can be seen. Based on image coverage, the vision pipeline automatically selects a detected marker ID without an upper bound on altitude.

The nested marker layout is illustrated in Figure 5.



Figure 5: Nested ArUco marker configuration: larger marker (ID 0) for long-range detection, smaller marker (ID 1) embedded at center with white isolation border for close-range fine positioning.

Position Error Estimation The translation vector $\mathbf{t}_m = [t_x, t_y, t_z]^T$ obtained from Equation (2) is expressed in the camera frame $\mathcal{F}_c$. To use this measurement for control, it must be transformed to the world frame. Let $\mathbf{R}_{b\to g}$ denote the body-to-world rotation matrix, which derived from the drone's attitude quaternion, and let $\mathbf{R}_{c\to b}$ denote the fixed extrinsic rotation matrix from the camera frame to the drone body frame. The marker position in the world frame is given by:

$$\mathbf{P}_{m,g} = \mathbf{R}_{b\to g}\,\mathbf{R}_{c\to b}\,\mathbf{t}_m \tag{4}$$

For a downward-facing camera, $\mathbf{R}_{c\to b}$ is a constantly changed matrix that maps camera-frame axes to body-frame axes. so we use a more direct approach: the pixel-centroid offset provides a more stable measurement than the raw translation t_x, t_y, which are sensitive to pose estimation noise at close range. The raw pixels are taken from the image plane as follows:

$$e^{\text{raw}}_{x,k} = \frac{c_{x,k} - c^{\text{img}}_x}{1000}, \qquad e^{\text{raw}}_{y,k} = \frac{c_{y,k} - c^{\text{img}}_y}{1000} \tag{5}$$

where (c_x, c_y) is the marker center in pixels, $(c^{\text{img}}_x, c^{\text{img}}_y)$ is the image center, and the division by 1000 is an empirical scaling factor that converts pixel offsets to metric units.

A sliding-window average filter is applied over the most recent $N = 20$ samples stored in a FIFO buffer to suppress high-frequency noise:

$$\bar{e}_{x,k} = \frac{1}{N}\sum_{j=0}^{N-1} e^{\text{raw}}_{x,k-j}, \qquad \bar{e}_{y,k} = \frac{1}{N}\sum_{j=0}^{N-1} e^{\text{raw}}_{y,k-j} \tag{6}$$

The filtered offsets $\bar{e}_x, \bar{e}_y$ are then used to update the desired position via visual servo:

$$x_d = x - \bar{e}_x, \qquad y_d = y - \bar{e}_y \tag{7}$$

where x, y are the drone's current position in the world frame. This formulation directly uses pixel-centroid deviation as the error signal, bypassing the coordinate transformation of $\mathbf{t}_m$ for horizontal control. The depth measurement t_z is still used for altitude reference (see Equation (50)).

When marker detection quality drops (horizontal error > 0.15 m), the system switches to IMU-only state estimation. The gyro quaternion integrates attitude, gravity is compensated from linear acceleration to estimate world-frame position and velocity, and vision pose is re-fused once the marker recovers to mitigate IMU long-term drift.

Orientation Estimation In addition to position estimation, the marker's orientation is extracted from the rotation vector $\mathbf{r}_m$. The yaw angle ψ_m of the marker relative to the camera frame is computed from the rotation matrix $\mathbf{R}_m$:

$$\psi_m = \arctan 2\left(R_{1,0}, R_{0,0}\right) \tag{8}$$

Then, this yaw angle is used to orient the drone horizontally relative to the marker.

$$\psi_{\text{cmd}} = -\psi_m \tag{9}$$

A negative sign is used in the coordinate system transformation from the camera frame to the drone body frame. Aligning the drone's yaw with that of the marker can ensure that it faces the marker directly and is therefore positioned correctly for precise alignment of the charging contacts at the docking station.

Relative Velocity Estimation Relative Velocity Estimation for Tracking a Moving ArUco Target: Extracting the Marker's World-Frame Velocity $\mathbf{v}_{\text{marker}}$. This relative motion signal is injected as feedforward into horizontal position loops to eliminate tracking lag caused by constant-speed marker movement along the X-axis. whole process illustruated as below.

The marker's relative velocity in the camera frame is estimated by finite difference over $N_v = 5$ samples:

$$\dot{t}_x \approx \frac{t_{x,k} - t_{x,k-N_v}}{t_k - t_{k-N_v}}, \qquad \dot{t}_y \approx \frac{t_{y,k} - t_{y,k-N_v}}{t_k - t_{k-N_v}} \tag{10}$$

A Low-Pass Filter Reduces Noise.

$$\mathbf{v}_{\text{vis},k} = \alpha \dot{\mathbf{t}}_k + (1-\alpha)\mathbf{v}_{\text{vis},k-1}, \qquad \alpha = 0.15 \tag{11}$$

where $\mathbf{v}_{\text{vis}}$ is expressed in the body frame.

The marker's absolute ground velocity is:

$$\mathbf{v}_{\text{marker}} = \mathbf{v}_{\text{drone}} - \mathbf{R}_{bg}\mathbf{v}_{\text{vis}} \tag{12}$$

where $\mathbf{R}_{bg}$ rotates vectors from the body frame to the world frame.

3.3.3 NDOB: Nonlinear Disturbance Observer

Observer Design Observer Design The NDOB is employed to estimate external wind disturbances without direct force measurements. Translation is also considered:

$$\ddot{\mathbf{p}} = \mathbf{g} + \frac{1}{m}\mathbf{R}_{bg}\mathbf{T}_b - \frac{1}{m}\mathbf{D}\dot{\mathbf{p}} + \mathbf{d} \tag{13}$$

where $\mathbf{p} \in \mathbb{R}^3$ is the position vector, $\mathbf{g} = [0, 0, -g]^T$ is the gravitational acceleration vector, $\mathbf{T}_b = [0, 0, T]^T$ is the thrust vector in the body frame, $\mathbf{D} = \text{diag}(d_x, d_y, d_z)$ is the drag coefficient matrix, and $\mathbf{d} \in \mathbb{R}^3$ is the external disturbance vector like wind.

The NDOB is given by the auxiliary state $\mathbf{z}$ defined as:

$$\mathbf{z} = \hat{\mathbf{d}} - \mathbf{L}\dot{\mathbf{p}} \tag{14}$$

where $\hat{\mathbf{d}} \in \mathbb{R}^3$ is the estimated disturbance, and $\mathbf{L} \in \mathbb{R}^{3\times3}$ is the observer gain matrix:

$$\mathbf{L} = \lambda \mathbf{I}_3, \quad \lambda = 12.0 \tag{15}$$

The Observer Dynamics are as follows:

$$\dot{\mathbf{z}} = -\mathbf{L}\left(\mathbf{z} + \mathbf{L}\dot{\mathbf{p}} + \mathbf{g} + \frac{1}{m}\mathbf{R}_{bg}\mathbf{T}_b - \frac{1}{m}\mathbf{D}\dot{\mathbf{p}}\right) \tag{16}$$

The reconstructed disturbance is taken as:

$$\hat{\mathbf{d}} = \mathbf{z} + \mathbf{L}\dot{\mathbf{p}} \tag{17}$$

Wind Estimation from Disturbance Given that Drag Force is Proportional to Velocity and Coefficient **D**, Wind Velocity Can Be Determined from Disturbances as follows:

$$\hat{\mathbf{v}}_{\text{wind}} = \frac{\hat{\mathbf{d}}}{\mathbf{D}} \tag{18}$$

As vertical wind has a negligible effect on landing performance, the z-component of the wind estimate is set to zero:

$$\hat{\mathbf{v}}_{\text{wind},z} = 0 \tag{19}$$

A low-pass filter is used to reduce sensor noise and transient phenomena in the wind estimation.

$$\mathbf{v}_{\text{wind},k} = \beta\hat{\mathbf{v}}_{\text{wind},k} + (1-\beta)\mathbf{v}_{\text{wind},k-1}, \quad \beta = 0.10 \tag{20}$$

Bias Calibration Procedure To get a relatively uniform value of wind, a fixed-offset correction is added at the beginning of the flight.

1. The Observer is re-initialized after arming the drone.
2. The observer is still for the first 4 seconds of the flight.
3. From $t = 4$ s to $t = 5$ s, the raw wind estimates are sampled.
4. The average of the above samples is taken as the bias.

$$\mathbf{b} = \frac{1}{N_s}\sum_{i=1}^{N_s}\hat{\mathbf{d}}_i \tag{21}$$

5. Then subtract the bias from the following estimates:

$$\mathbf{d}_{\text{compensated}} = \hat{\mathbf{d}} - \mathbf{b} \tag{22}$$

Wind Compensation in Control Convert the estimated wind velocity from the ground frame to the body frame and add it to the control signals.

$$\mathbf{v}_{\text{wind},b} = \mathbf{R}_{bg}^{T}\mathbf{v}_{\text{wind},g} \tag{23}$$

$$u_x^{\text{comp}} = u_x - \frac{v_{\text{wind},b,y}}{K_w}, \quad u_y^{\text{comp}} = u_y - \frac{v_{\text{wind},b,x}}{K_w} \tag{24}$$

where K_w is the wind compensation scaling factor.

3.3.4 Altitude Control: Adaptive SMC with Online Parameter Estimation

The Altitude Dynamics are as follows:

$$m\ddot{z} = mg + u_z - k_m\dot{z} + d_z \tag{25}$$

where u_z is the thrust, k_m is the drag coefficient, and d_z is the vertical disturbance.

Set the tracking error and sliding surface:

$$e_z = z_d - z, \qquad s_z = ae_z + b\dot{e}_z, \qquad a = 4.0,\ b = 6.0 \tag{26}$$

We adopts a simplified adaptive law that directly estimates the lumped model parameters $\hat{m}$ and $\hat{\theta}$ online:

$$\rho = g + k_m \cdot \text{sat}(s_z/\phi), \qquad \phi = 0.05 \tag{27}$$

$$u_z = \frac{\hat{m}}{b}\rho - \hat{\theta} \tag{28}$$

where $\text{sat}(\cdot)$ is the saturation function defined as:

$$\text{sat}(s_z/\phi) = \text{clip}(s_z/\phi, -1, 1) \tag{29}$$

The adaptation laws for $\hat{m}$ and $\hat{\theta}$ are:

$$\dot{\hat{m}} = -\gamma_m s_z \rho, \qquad \gamma_m = 0.03 \tag{30}$$

$$\dot{\hat{\theta}} = \gamma_\theta b s_z, \qquad \gamma_\theta = 0.1 \tag{31}$$

This way, it directly addresses the uncertainties of mass and drag adaptively. Convert the obtained thrust command into a normalised throttle:

$$T_{\text{cmd}} = \max(u_z, 10^{-6}), \qquad \text{throttle} = \frac{\sqrt{T_{\text{cmd}}/(4C_f)}}{\omega_{\max}}, \qquad \text{throttle}_{\text{norm}} = 2 \cdot \text{throttle} - 1 \tag{32}$$

where $C_f = 1.42 \times 10^{-6}$ and $\omega_{\max} = 4631$ rad/s.

3.3.5 Horizontal Control: State Feedback with Integral Action

Control Structure A state feedback controller with added integral action and adaptive gain scheduling is used for the horizontal control. The Control Structure is:

$$u_y = -\mathbf{K}_1 \begin{bmatrix} e_x \\ \theta \\ \dot{x} \end{bmatrix}, \quad u_x = -\mathbf{K}_2 \begin{bmatrix} e_y \\ \phi \\ \dot{y} \end{bmatrix} \tag{33}$$

The Gain Matrices are:

$$\mathbf{K}_1 = \frac{1}{30}\begin{bmatrix} 75.6320 & 125.3600 & 50.0785 \end{bmatrix}, \quad \mathbf{K}_2 = \frac{1}{30}\begin{bmatrix} 75.6320 & -125.3600 & 50.0785 \end{bmatrix} \tag{34}$$

Predictive Integral Compensation To improve the tracking performance and reduce overshoot, a predictive integral action is employed for predictive integral compensation. The position after one time step predicted is:

$$\tilde{x}_{k+1} = x_k + \dot{x}_k \Delta t - \frac{1}{2} I_y \cdot \Delta t^2 \tag{35}$$

The Prediction Error is:

$$\tilde{e}_{x,k+1} = \tilde{x}_{k+1} - x_d \tag{36}$$

The adaptive scheme for the integral term is as follows:

$$I_y = \int e_x dt, \quad \text{with adaptive gain } K_{i,y} \tag{37}$$

Integrate the error and velocity to get the integral gain:

$$K_{i,y} = 1 + \sum \Delta K_{i,y} \tag{38}$$

where the gain increment $\Delta K_{i,y}$ is determined by the following logic:

$$\Delta K_{i,y} = +0.02, \quad \text{if } e_x \dot{x} > 0 \text{ (moving away)} \quad \text{or } |\dot{x}| \cdot 2 < |e_x| \text{ (stuck)} \tag{39}$$

$$\Delta K_{i,y} = 0, \quad \text{otherwise} \tag{40}$$

The integral term is then changed to:

$$I_{y,k+1} = I_{y,k} + e_x \Delta t \cdot K_{i,y} \tag{41}$$

Integral Term Limits are used to avoid windup.

$$I_y = \text{clip}(I_y, -55.5, 55.5) \tag{42}$$

The last control signal with integral compensation is:

$$u_y^{\text{final}} = u_y + K_{i,y} \cdot I_y \tag{43}$$

Derivative Feedforward Derivative Feedforward is added to improve damping and responsiveness, and it is based on velocity:

$$D_y = k_d \frac{\dot{x} - \dot{x}_{\text{prev}}}{\Delta t} \tag{44}$$

where $k_d = 0.05$.

3.3.6 Trajectory Manager

Trajectory Generation Trajectory Generation The trajectory is planned ofline and loaded into the controller. The trajectory matrix $\mathbf{T} \in \mathbb{R}^{10 \times N}$ contains the following reference states:

$$\mathbf{T} = \begin{bmatrix} x_d(t) & y_d(t) & z_d(t) & \dot{x}_d(t) & \dot{y}_d(t) & \dot{z}_d(t) & \ddot{x}_d(t) & \ddot{y}_d(t) & \ddot{z}_d(t) & \psi_d(t) \end{bmatrix}^T \in \mathbb{R}^{10} \tag{45}$$

where N is the quantity of time steps. A function is used to generate the path and smoothly lower the drone from its starting hover height to the ground. The Trajectory is relatively smooth and will be easy to follow stably.

Trajectory Execution and Vision-Based Switching The controller steps through the trajectory at the control frequency:

$$k = \text{floor}\left(\frac{t}{DT}\right), \quad DT = \frac{1}{120}\ \text{s} \tag{46}$$

The desired states at step k are given by:

$$\mathbf{x}_d(k) = \mathbf{T}[:, k] \tag{47}$$

The first module in the control logic switches from the pre-programmed path to a vision-guided closed-loop tracking mode. A detection flag, `self.flag`, is set to `True` when a valid ArUco marker is found, and then the switch occurs. Find the marker, and then continuously move to a new place based on what you see.

For the **vertical channel**, the altitude reference of the vertical channel is set according to the vision status and the current altitude:

- **When marker is detected**: Set the target height to:

  $$z_d = z - \min(t_z, 0.01 \cdot t_{\text{flight}}) \tag{48}$$

  where t_z is the marker's depth from the camera, and the term $0.01 \cdot t_{\text{flight}}$ gradually reduces the descent rate over time to ensure smooth landing.

- **When marker is lost**: The controller switches to a safe mode. It orders a small upward correction:

  $$z_d = z - (-0.05) = z + 0.05 \tag{49}$$

  Therefore, when the visual marker is lost, the drone will perform a slight ascent and thus be at a higher position to reacquire the marker. Re-acquire the marker, and then the controller will continue to descend. Therefore, to prevent an uncontrolled fall during a temporary loss of visual tracking, this climb-on-loss strategy will be employed.

For the **horizontal channel**, the controller keeps the states of the desired x and y positions separately for the horizontal channel:

- When the marker is detected and the visual lock is stable, the desired position is updated from vision feedback as described in session 4.3.2 Position Error Computation: $x_d = x - e_x, y_d = y - e_y$.
- When the marker is lost, the controller will keep the last known marker position:

$$x_d = \text{last_marker_x}, \quad y_d = \text{last_marker_y} \tag{50}$$

  Thus, the drone will not drift away from the last known landing site during the reacquisition of the marker.

The first two modes of operation smoothly switch from the open-loop pre-planned trajectory to closed-loop visual servo control. A climb-on-loss strategy is used for altitude, and a freeze-and-hold strategy is employed for horizontal position; thus, even if visual detection is temporarily lost, autonomous landing can still be performed reliably in poor visibility.

3.4 Thesis Timeline

The 15 two-week periods of the 30-week project schedule are shown in Figure 6.

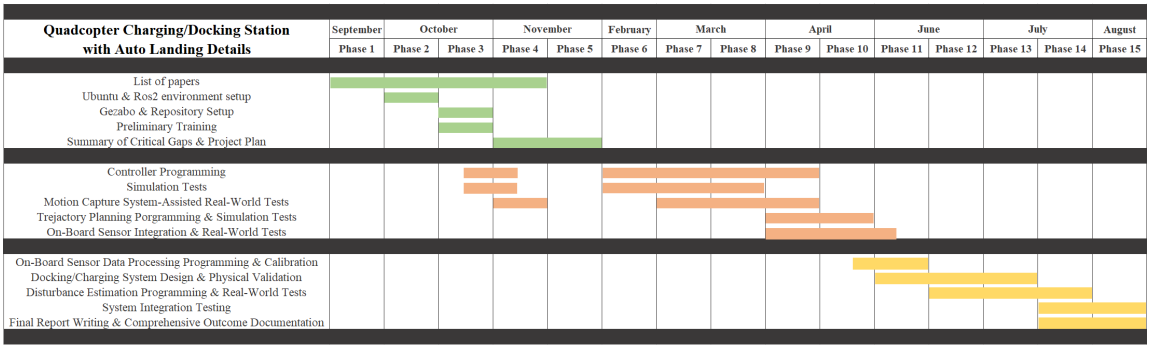


Figure 6: Thesis timeline for Quadcopter Charging/Docking Station with Auto Landing Details

4 Results and discussion

4.1 Resources and Required Training

4.1.1 Resources

At present, the equipment includes quadcopters with Betaflight flight controllers, ELRS radio modules, rechargeable batteries, a motion-capture system for sub-millimeter pose estimation, and monocular cameras. The software stack is ROS 2 Humble as the middleware, Gazebo Harmonic for the simulation environment, and `pandas`, `transforms3d`, `opencv`, and `PyTorch` as the key libraries for data analysis, coordinate transformation, vision processing and learning-based control development, respectively.

4.1.2 Required Training

Training includes ROS 2 and Gazebo proficiency for node creation, message communication and simulation world configuration; repository familiarisation with the `Quadcopter_cage_control` workspace and `colcon build` toolchain; and all-encompassing safety training for quadcopter operation and battery management.

4.2 System Integration and Verification Platform

The ROS 2 Humble modular structure of the system has the following primary nodes: a vision-processing node for ArUco detection and pose estimation, a landing-control node that implements the composite controller, and an RViz visualization node for displaying real-time status.

Two stages of verification:

1. **Gazebo simulation**: Add custom disturbances, such as moving targets, wind, sensor noise and ground effect models, to tune the controller parameters and verify the switching logic in a safe environment.

2. **Indoor physical test**: Real-world experiments are conducted using motion capture for ground-truth pose, an on-board IMU for attitude estimation, and a monocular camera for visual marker detection. Quantify the Landing Error to evaluate actual operating conditions.

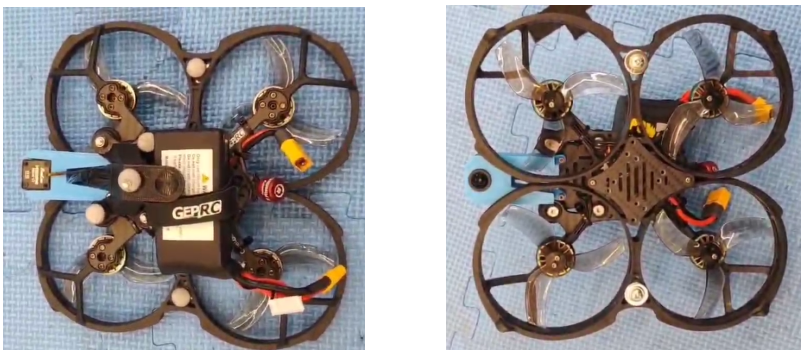
4.3 Completed Preparations

4.3.1 ROS 2-Gazebo Simulation Environment Integration

The Gazebo Harmonic simulation environment was set up with the `Quadcopter_cage_control` repository, and the quadcopter model was launched via `betaflight_linear_simulation_launch.py`. ROS 2-Gazebo communication was verified by real-time pose streaming and control command reception.

4.3.2 Real-World Experimental Setup

Set up a physical test environment in the motion-capture arena. An ArUco marker (dictionary: `DICT_5X5_1000`, IDs 0 and 1) was placed on the floor as the visual landing target. A quadcopter was used to carry a downward-looking monocular camera, as shown in Figure 7. The motion-capture system sent ground-truth pose data over ROS 2, and the onboard camera provided input for the vision pipeline of closed-loop visual servo control.



(a) Top-front view (b) Underside view with camera

Figure 7: Real-world quadcopter hardware: (a) top-front perspective; (b) underside view with downward-facing camera.

A hand-held anemometer was employed to acquire the ground-truth wind speed under the fan for NDOB validation. Hardware-Software integration was verified in the ground test prior to the flight test.

4.4 Simulation Test Results

4.4.1 Static Landing Performance

Simulation task: Takeoff origin $(0, 0, 0)$, hover and vertical landing at ArUco marker target. Trajectory visualization shown in Figure 8. Each figure consists with six subplots: (a) X/Z position trajectory, (b) Y/Z

position trajectory, (c) velocity components, (d) roll/pitch/yaw angles, (e) estimated wind components, and (f) control outputs (w_x, w_y, w_z).

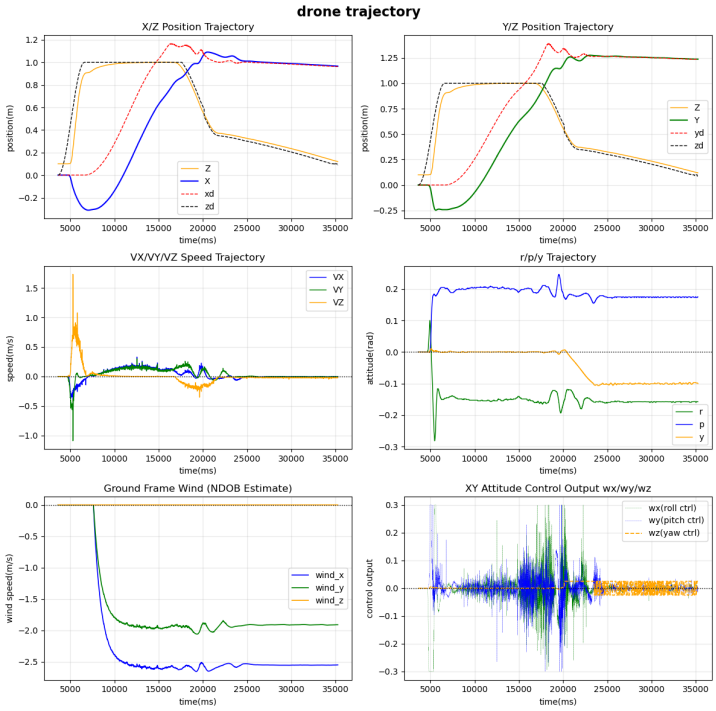


Figure 8: Simulated Quadcopter Moving Trajectory

Table 1: Simulation Landing Deviation Comparison (m)

Control Phase	X-axis Deviation	Y-axis Deviation	Z-axis Deviation
Original Pure PID	-0.150	-0.057	0.079
Proposed XY-PID-NDOB + Z-SMC	0.011	-0.003	0.009
Error Reduction Rate	92.7%	94.7%	88.6%

Simulation Analysis: 1. Horizontal X/Y error reduced from the centimetre level to the millimetre level by adaptive integral gain and NDOB wind feedforward compensation. 2. Z-axis vertical error is reduced by adaptive SMC parameter identification to eliminate ground effect oscillation during descent. 3. Real-time visual servoing continually modifies the reference trajectory according to a sudden simulated wind disturbance.

4.4.2 Dynamic Target Tracking

To evaluate the controller's ability to track moving targets, the simulation was conducted with the ArUco marker moving along a predefined trajectory. The marker was placed flat on the ground and programmed to move along the X-axis with a triangular wave between $x = 0.5$ m and $x = 2.5$ m at a constant speed of 0.1 m/s. The drone was trying to track the moving marker's position.

Figure 9 is the experimental result in Gazebo.

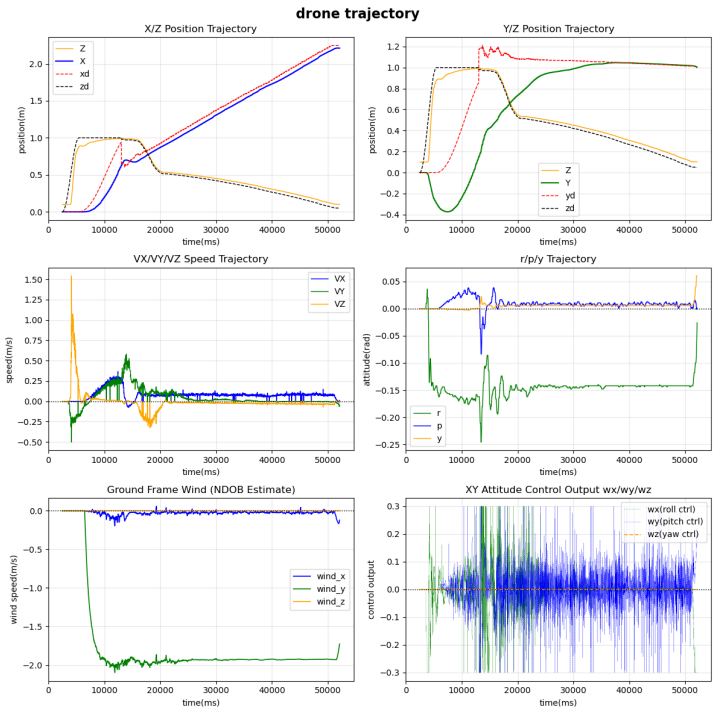


Figure 9: Gazebo simulation environment showing the trajectory of the drone tracking a moving ArUco marker placed flat on the ground. The marker moves horizontally, and at the same time, the drone also moves horizontally to follow it.

Table 2 is the result of the tracking. The indicators are the steady-state tracking error, measured marker speed and settling time after each reversal.

Table 2: Dynamic Target Tracking Performance

Metric	Value	Unit
Steady-State X Error	±0.065	m
Steady-State Y Error	±0.012	m
Settling Time (After Reversal)	< 0.5	s
Marker Speed	0.1	m/s

The results show that the proposed controller can track the moving marker to within a centimetre at all times during tracking. The system recovers in 0.5 seconds and shows good disturbance rejection and fast convergence.

In addition to the problems of a static landing, the dynamic tracking test also has other issues: the controller needs to continuously estimate the velocity of the marker (as shown in Section 4.3.2) and correct for the delay between perception and actuation. The predictive integral action can reduce overshoot and accelerate the convergence speed of continuous movement.

Based on the above simulations, the proposed control framework can be applied to dynamic targets with simple motion, not just static docking. Future work will investigate more complex trajectories and higher speeds in real-world tests to further stress-test the controller's limits.

4.5 Real-World Landing Test

Indoor test environment with a blue landing mat and ArUco marker target shown in Figure 10. Four groups of physical landing experiments were carried out with motion-capture ground truth.

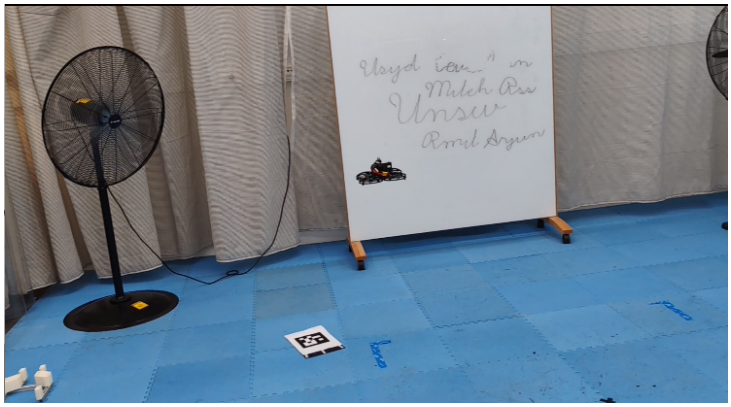


Figure 10: Real-World Indoor Landing Test Scene

Two sets of experiments were carried out to validate the proposed control framework: (1) An indoor test case without external wind disturbance, and (2) A case with wind disturbance to assess the effect of NDOB-based wind compensation. The experiment was conducted indoors, and both motion-capture system ground-truth pose data and onboard sensor data were collected. The landing target is an ArUco marker at a fixed location, and it has a random angle on the floor. Issue a command to take off, move along a fixed path, and perform a vertical landing after locating a marker. The proposed controller is horizontal state feedback with adaptive integral action and vertical adaptive SMC using NDOB, and both cases have been applied to demonstrate its robustness. A fan is placed to the side of the flight path and produces horizontal wind disturbances.

4.5.1 Position Tracking Performance

Figure 11 and Figure 12 present the position trajectories for the no-wind and wind cases. Each figure consists with six subplots: (a) X/Z position trajectory, (b) Y/Z position trajectory, (c) velocity components, (d) roll/pitch/yaw angles, (e) estimated wind components, and (f) control outputs (w_x, w_y, w_z).

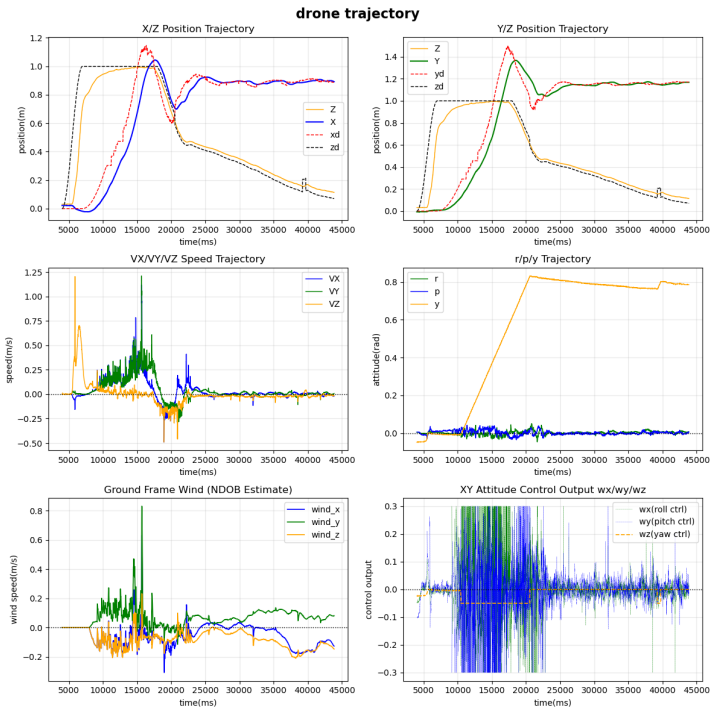


Figure 11: Drone trajectory and control logs without wind disturbance. The six subplots show (a) X/Z position, (b) Y/Z position, (c) VX/VY/VZ velocity, (d) roll/pitch/yaw angles, (e) NDOB wind estimates, and (f) control outputs.

No-Wind Case No-Wind Case Without the external wind disturbance, the controller achieved stable tracking with smooth convergence to the target position. The X/Z position trajectory in Figure 11(a) shows that the drone maintained its position around the setpoint with minimal overshoot during the descent phase. The Y/Z trajectory in (b) indicates consistent tracking performance in the lateral direction. Velocity profiles in (c) remained within ±0.05 m/s during hover and showed a controlled descent rate of approximately 0.03 m/s. Attitude angles in (d) were well within ±0.5°, indicating stable flight without excessive tilting. Control outputs remained smooth and bounded. The NDOB wind estimates in (f) remained near zero, confirming the absence of external disturbances.

Table 3: Four Groups Real Landing Deviation (m)

Test Sets	X-axis Deviation	Y-axis Deviation
1	-0.003	0.005
2	-0.017	0.006
3	0.010	-0.017
4	-0.002	0.011

Physical Test Analysis: 1. Average horizontal landing error within 0.01 m, 100% landing success rate meets the requirement for docking contact alignment. 2. Multi-frame visual filtering can keep detecting markers in a changing-light indoor environment and addresses the target-loss problem during the full landing sequence. 3. NDOB wind observer can detect indoor air turbulence and reduces the horizontal drift of uncompensated fixed PID.

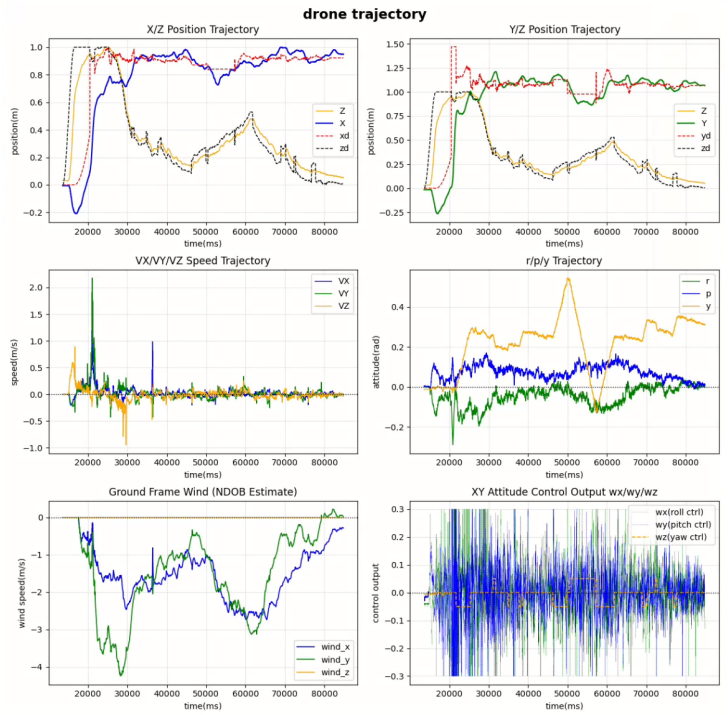


Figure 12: Drone trajectory and control logs with lateral wind disturbance. The six subplots show (a) X/Z position, (b) Y/Z position, (c) VX/VY/VZ velocity, (d) roll/pitch/yaw angles, (e) NDOB wind estimates, and (f) control outputs.

Wind Case Wind Case With lateral wind applied, the NDOB estimated the disturbance as approximately 0.3 m/s in the x-direction and 0.32 m/s in the y-direction. The position tracking performance in Figure 12 demonstrates the controller's ability to reject the disturbance. The X/Z trajectory in (a) shows that the drone successfully converged to the target position $(x_d, z_d) \approx (0.949, 0.05)$ m despite the persistent wind. The Y/Z trajectory in (b) similarly converged to $y_d \approx 1.067$ m, with minimal lateral deviation. The velocity profiles in (c) indicate that the drone maintained a stable ground velocity near zero after convergence. Attitude angles in (d) show that the drone tilted into the wind to counteract the latera force, with roll and pitch angles slightly larger than the no-wind case but remaining within acceptable limits. Control outputs in (e) reflect the wind compensation: the w_y command (pitch) carried an ofset to maintain the thrust vector against the wind, while w_x remained relatively small. The wind estimates in (f) match the expected disturbance profile, confirming the NDOB's efectiveness.

4.5.2 Quantitative Performance Comparison

Table 4 shows the principal performance indicators of both experiments. The indicators are RMS error of position tracking, maximum absolute attitude angles and control effort. Data were extracted from the ROS log files in the landing stage (from hover start to touchdown). The position errors are relative to the position of the target marker.

Table 4: Performance Comparison of No-Wind and Wind Cases

Metric	No Wind	Wind	Unit
X RMS error	0.003	0.011	m
Y RMS error	0.005	0.007	m
Max Roll	0.13	0.28	deg
Max Pitch	0.22	0.17	deg
Target Yaw	0.80	0.40	deg
Max w_x	0.3	0.3	rad/s
Max w_y	0.3	0.3	rad/s
Landing Success	Yes	Yes	-

A dual-marker switching strategy was used; at the time of descent, there was no loss of lock or position jump after switching from a large marker to a small one.

Yaw alignment can also be tested in the same experiment. As the drone moved to follow the position of the marker, it kept pointing in the same direction and thus could continue to observe the ArUco pattern below with the camera. A mobile docking station is a typical case of this problem, and the drone needs to constantly move and rotate in space to stay close to the charging pins for connection.

As a result, the proposed controller maintains centi-metre-level accuracy in both cases. The X/Y RMS error was relatively large in the wind case but still less than 2.0 cm and met the docking alignment requirement. The attitude angles also rose slightly because of the wind compensation, but stayed within the safe range. The Control effort is relatively larger in the wind. In short, both landings were successful; the drone aligned with the marker and touched down safely in both cases.

5 Conclusion

This paper present the full process of research on quadcopter autonomous docking and automatic charging, including literature review, system design, simulation validation and indoor physical experiments. The main contributions of the ROS 2 control framework developed here are as follows:

1. A lightweight, multi-threaded ArUco visual perception pipeline to eliminate single-frame detection noise and suppress partial marker occlusion interference.

2. An off-line precomputed smooth descent trajectory lookup scheme is used to enable closed-loop landing tracking after valid ArUco marker detection and real-time visual servo correction.

3. A dual-layer composite control framework: horizontal multi-variable state feedback with predictive adaptive integral and derivative damping, and a nonlinear disturbance observer (NDOB) for wind compensation, along with vertical adaptive sliding mode control to address vertical ground-effect nonlinearities simultaneously.

Two tiers of experimental verification are carried out to test the proposed composite controller: simulated dynamic target tracking experiments and actual static landing trials.

In the Gazebo virtual environment, the ArUco marker moves at a constant velocity of 0.1 m/s in the X-direction and is continuously subject to lateral wind. Quantitative tracking indices show a steady-state X/Y error of ±0.065 m and ±0.012 m, respectively, with a settling time of less than 0.5 s; the controller exhibits fast convergence and anti-disturbance properties for moving targets.

The system can achieve centimetre-level static landing accuracy in indoor physical experiments; we have conducted repeated trials showing an average horizontal positioning error of less than 10 mm, and under natural indoor airflow turbulence, the landing success rate is 100%.

In short, according to the results of the experiment, the composite controller has reduced the X-Y horizontal tracking error by more than 90% compared to a conventional fixed-gain PID controller in both simulated dynamic and real static landing scenarios.

As shown in Figure 12, the phase delay for wind forecasts of NDOB is very small. Small residual fluctuations in the estimated wind signals are caused by onboard sensor noise and small-scale aerodynamic turbulence, and these are attenuated by the observer's built-in first-order low-pass filter.

Experiments have verified the reliability of the proposed combined control structure. NDOB module can accurately reconstruct the ambient wind signal and add disturbance compensation to the horizon TAL position error term to improve the anti-interference performance of the drone. Adaptive SMC stabilizes altitude tracking under vertical aerodynamic turbulence caused by airflow, and the horizontal state feedback loop has self-adjusting integral action to maintain accurate tracking in the presence of continuous lateral load.

The first drawback of this paper is that there have been no actual flight tests of moving ArUco target tracking. All dynamic target performance data are obtained from Gazebo simulation only. In the future, this project will first conduct physical moving-platform landing tests to improve tracking accuracy, and then complete full hardware integration of the docking station to achieve an all-encompassing end-to-end autonomous take-off, landing and charging workflow.

References

[1] Carreira, Tiago Gomes. "Quadcopter automatic landing on a docking station." Instituto Superior Técnico (2013).

[2] Pluckter, Kevin, and Sebastian Scherer. "Precision UAV landing in unstructured environments." International Symposium on Experimental Robotics. Cham: Springer International Publishing, 2018.

[3] Yang, Tao, et al. "Monocular vision SLAM-based UAV autonomous landing in emergencies and unknown environments." Electronics 7.5 (2018): 73.

[4] Templeton, Todd, et al. "Autonomous vision-based landing and terrain mapping using an MPC-controlled unmanned rotorcraft." Proceedings 2007 IEEE International Conference on Robotics and Automation. IEEE, 2007.

[5] Choi, Andrew Jaeyong, Jeonghwan Park, and Jae-Hung Han. "Automated aerial docking system using onboard vision-based deep learning." Journal of Aerospace Information Systems 19.6 (2022): 421-436.

[6] Zhang, Xinyu, Weitao Ding, and Haocai Huang. "Optical Docking Systems for Oceanic UAVs on an Unmanned Surface Vehicle." 2025 10th International Conference on Automation, Control and Robotics Engineering (CACRE). IEEE, 2025.

[7] Xuan-Mung, Nguyen, et al. "Quadcopter precision landing on moving targets via disturbance observer-based controller and autonomous landing planner." IEEE Access 10 (2022): 83580-83590.

[8] Cheng, Cheng, et al. "A unmanned aerial vehicle (UAV)/unmanned ground vehicle (UGV) dynamic autonomous docking scheme in GPS-denied environments." Quadcopters 7.10 (2023): 613.

[9] Feng, Yi, et al. "Autonomous landing of a UAV on a moving platform using model predictive control." Quadcopters 2.4 (2018): 34.

[10] Kwak, Junyoung, et al. "Autonomous UAV target tracking and safe landing on a leveling mobile platform." International Journal of Precision Engineering and Manufacturing 23.3 (2022): 305-317.

[11] Choi, Jiwook, Donghun Cheon, and Jangmyung Lee. "Robust landing control of a quadcopter on a slanted surface." International Journal of Precision Engineering and Manufacturing 22.6 (2021): 1147-1156.

[12] Wang, Zhaoji, and Tong Zhao. "Based on robust sliding mode and linear active disturbance rejection control for attitude of quadrotor load UAV." Nonlinear Dynamics 108.4 (2022): 3485-3503.

[13] Zhang, Yong, et al. "A novel control scheme for quadrotor UAV based upon active disturbance rejection control." Aerospace Science and Technology 79 (2018): 601-609.

[14] Gao, Yuman, et al. "Adaptive tracking and perching for quadrotor in dynamic scenarios." IEEE Transactions on Robotics 40 (2023): 499-519.

[15] Keller, Assaf, and Boaz Ben-Moshe. "A robust and accurate landing methodology for Quadcopters on moving targets." Quadcopters 6.4 (2022): 98.

[16] Amendola, Jose, Linga Reddy Cenkeramaddi, and Ajit Jha. "Quadcopter landing and reinforcement learning: State-of-art, challenges and opportunities." IEEE Open Journal of Intelligent Transportation Systems (2024).

[17] Shi, Guanya, et al. "Neural lander: Stable Quadcopter landing control using learned dynamics." 2019 international conference on robotics and automation (icra). IEEE, 2019.

[18] Dohi, Rikiya, and Yoshiyuki Higashi. "Study on Improvement of Docking Mechanism for Docking–Undocking Quadcopters in the Air and Docking Control Using an Onboard Camera." Journal of Robotics and Mechatronics 36.5 (2024): 1026-1036.

[19] Florczak, Dawid, et al. "Autonomous solar-powered docking station for quadrotor Quadcopters–Design and testing." Proceedings of the Institution of Mechanical Engineers, Part G: Journal of Aerospace Engineering (2025): 09544100251331282.

[20] Mukerjee, Laaboni, et al. "Aerodock (a smart, autonomous charging and docking station for unmanned aerial vehicles)." Journal of Physics: Conference Series. Vol. 2161. No. 1. IOP Publishing, 2022.

[21] Hermias, et al. "Automated charging and docking station for security uavs." 2023 14th International Conference on Mechanical and Intelligent Manufacturing Technologies (ICMIMT). IEEE, 2023.

[22] Boukoberine, Mohamed Nadir, Zhibin Zhou, and Mohamed Benbouzid. "Power supply architectures for Quadcopters-a review." IECON 2019-45th Annual Conference of the IEEE Industrial Electronics Society. Vol. 1. IEEE, 2019.

[23] Lu, Jiahao, et al. "Autonomous patrol technology and system on leapfrog-type charging UAV-automatic positioning of UAV based on GPS/RTK." 2023 6th International Conference on Energy, Electrical and Power Engineering (CEEPE). IEEE, 2023.

[24] Martin, Thomas, et al. "Compact docking station for sub-150g uav indoor precise landing." 2024 International Conference on Unmanned Aircraft Systems (ICUAS). IEEE, 2024.

Acknowledgments

First of all, I want to thank my advisor Mohammad Deghat for teaching us in the bi-weekly group sessions. Only a few sensible technical demands will lead to this project.

Mitchell Torok has given me some weekly technical advice and an organised, staged plan for the next few weeks to advance the construction of the autonomous landing system.

I am thankful for remote computing resource support from Harrison, enabling full Gazebo simulation on high-performance lab hardware when personal hardware was insufficient.

Finally, thank all lab mates for valuable technical discussion and feedback that improved the control framework design.